Industr.IA

E-book

IA Generativa: Camada Decisória

(decisões correlatas)

Curso base de IA | Unidade M1.C3

Prof.: Arthur Nascimento



SUMÁRIO

1. GenAI versus ML clássico 3
2. Modelo aberto versus API gerenciada 5
3. In-context learning versus fine-tuning 8
4. RAG versus contexto fechado 11
5. Alocação de custo e fechamento decisório 13
6. Conclusão 15

M1.C3

1. GenAI versus ML clássico

Esta seção pertence à camada decisória e tem por objetivo que você aplique a heurística GenAI versus ML clássico a um cenário industrial, justificando a escolha em termos de custo, latência, reprodutibilidade, precisão e disponibilidade de dados. Aqui o curso deixa de descrever arquitetura e passa a ensinar a decidir entre alternativas.

A primeira decisão da lente criativa não pergunta qual tecnologia é mais avançada. Pergunta qual classe de solução resolve a tarefa ao menor custo de risco. Tratar IA Generativa como o passo seguinte e mais sofisticado em relação ao Machine Learning clássico inverte a ordem correta do raciocínio: a tarefa decide, não a moda. Por isso a decisão se ancora em uma pergunta concreta sobre a saída esperada.

O custo de risco que orienta essa decisão tem componentes concretos. Há o risco de erro, quando a saída incorreta gera retrabalho ou prejuízo; o risco de custo, quando a operação em escala consome orçamento desproporcional ao valor entregue; o risco de manutenção, quando a solução exige cuidado contínuo que a equipe não tem como sustentar; e o risco de conformidade, quando a saída precisa ser auditável e o modelo escolhido não permite. A heurística GenAI versus ML clássico é, no fundo, uma forma de minimizar a soma desses riscos para a tarefa em jogo, e não de maximizar a sofisticação técnica.

A pergunta âncora separa dois regimes. Quando a tarefa exige sintetizar um artefato aberto, como um texto livre, um trecho de código, uma imagem ou um resumo, a saída não tem rótulo fixo a prever, e a síntese é a própria tarefa: a indicação pende fortemente para GenAI. Quando a tarefa exige produzir um rótulo, um valor ou uma estrutura a partir de uma entrada bem definida, como classificar um defeito, prever uma demanda ou agrupar registros, existe um alvo discreto e mensurável, e o Machine Learning clássico costuma ser o ponto de partida natural.

> Numa linha de manufatura, uma equipe precisa classificar chamados de manutenção em categorias já conhecidas, com milhares de exemplos rotulados acumulados no histórico. Um classificador baseado em gradient boosting resolve a tarefa com acurácia equivalente à de um modelo de linguagem grande, responde em poucos milissegundos e custa uma fração por inferência. O LLM só se justificaria se a tarefa passasse a exigir a redação da resposta ao técnico, e não apenas a etiqueta da categoria.

Confirmada a natureza da saída, a decisão se afina pelos cinco eixos que estruturam a matriz T1.1. O eixo de custo contrasta o gasto por inferência: um classificador clássico, depois de treinado, custa quase nada por chamada, enquanto uma chamada a um LLM

consome tokens ou tempo de GPU a cada execução. O eixo de latência opõe a resposta em milissegundos de um modelo pequeno aos tempos de centenas de milissegundos a alguns segundos de uma geração.

O eixo de reprodutibilidade favorece o ML clássico: um modelo determinístico produz a mesma saída para a mesma entrada e é auditável, ao passo que uma geração varia com temperatura, versão do modelo e contexto, o que dificulta auditoria e testes de regressão. O eixo de precisão é o que mais surpreende: em tarefas tabulares estreitas, um modelo discriminativo bem ajustado frequentemente iguala ou supera um LLM, de modo que a vantagem da GenAI aparece sobretudo em tarefas de linguagem aberta. O eixo de disponibilidade de dados inverte o jogo no início de um projeto: o ML clássico exige um conjunto rotulado representativo, enquanto a GenAI opera com poucos exemplos ou nenhum, oferecendo uma vantagem real de partida a frio.

Os cinco eixos raramente pesam igual. A tarefa em jogo define quais dominam: um sistema de apoio à decisão clínica põe precisão e reprodutibilidade acima de custo; um assistente interno de baixo risco tolera variabilidade em troca de rapidez de entrega; um serviço de alto volume faz o custo por inferência dominar os demais. Antes de comparar alternativas, vale ordenar os eixos por importância para o caso concreto, porque é essa ordenação que transforma a matriz T1.1 de um checklist genérico em um argumento específico. Dois cenários podem chegar a desfechos opostos com os mesmos dados, apenas porque pesam os eixos de forma diferente, e isso é legítimo desde que explicitado.

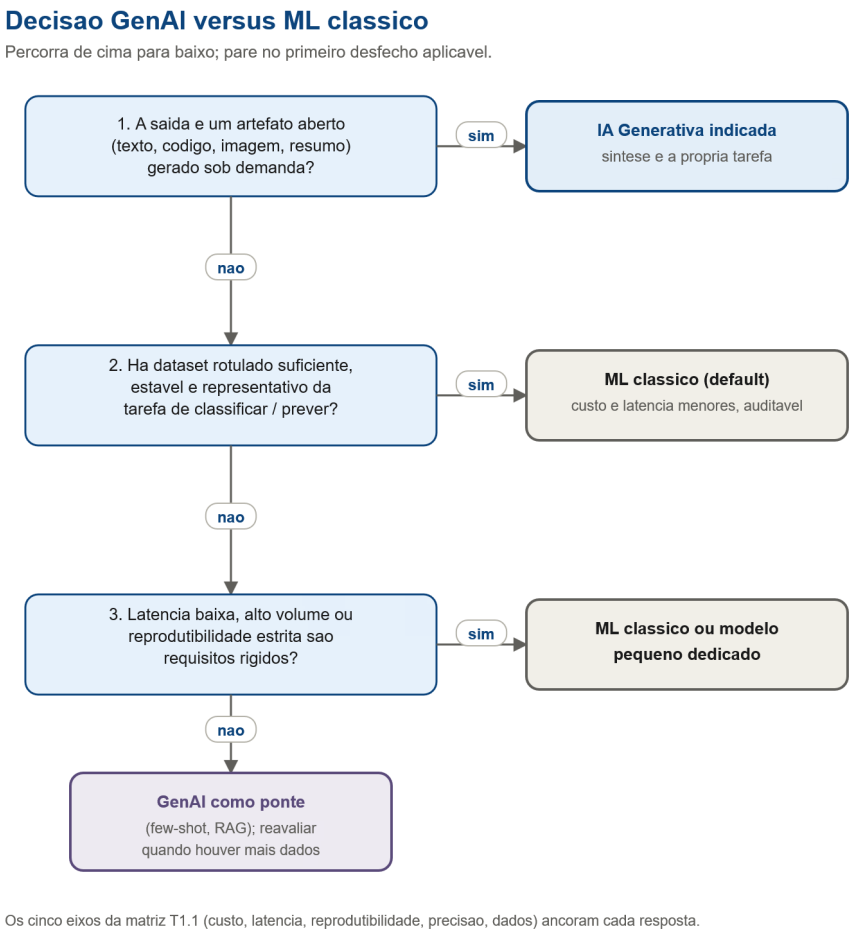
> ⚠ ATENÇÃO
>
> A confusão decisória mais frequente nesta etapa é tratar a GenAI como default moderno, supondo que um LLM é sempre a versão superior de qualquer solução. Essa suposição ignora que, sob paridade de precisão, o modelo clássico costuma vencer em custo, latência e auditabilidade. A consequência prática é um sistema mais caro, mais lento e menos reprodutível do que o necessário, defendido pela novidade e não pela adequação à tarefa.

Vale insistir no eixo de precisão, porque é onde a intuição mais erra. Em muitas tarefas tabulares, um modelo discriminativo bem ajustado e um LLM chegam a níveis de acurácia próximos; quando isso acontece, a precisão deixa de ser o critério de desempate, e a decisão se resolve nos outros quatro eixos, em geral a favor do modelo clássico, mais barato, mais rápido e mais auditável. A lição não é que o ML clássico domina sempre, e sim que, sob paridade de precisão, os custos assimétricos decidem. Reconhecer a

paridade quando ela existe evita pagar caro por uma diferença que não se materializa.

A árvore de decisão a seguir condensa a heurística em três perguntas encadeadas. Percorra de cima para baixo e pare no primeiro desfecho aplicável; cada resposta deve ser justificada com referência explícita aos cinco eixos.

Figura 1 - Árvore de decisão GenAI versus ML clássico, com três perguntas encadeadas e os desfechos correspondentes.



Fonte: autor.

Para cego ver: A imagem apresenta um fluxograma intitulado "Figura 4.
Árvore de decisão GenAI versus ML clássico, com três perguntas encadeadas e os desfechos correspondentes". O diagrama orienta a escolha entre Inteligência Artificial Generativa (GenAI) e Aprendizado de Máquina clássico (ML clássico) por meio de três perguntas sequenciais. A primeira pergunta é se a saída esperada é um artefato aberto, como texto, código, imagem ou resumo gerado sob demanda. Se a resposta for sim, a recomendação é utilizar IA Generativa, pois a síntese do conteúdo faz parte da própria tarefa. Se a resposta for não, o fluxo segue para a segunda pergunta: se existe um conjunto de dados rotulado, suficiente, estável e representativo para realizar tarefas de classificação ou previsão. Em caso afirmativo, a indicação é utilizar ML clássico, destacando menor custo, menor latência e maior auditabilidade. Caso contrário, o fluxo avança para a terceira pergunta, que verifica se baixa latência, alto volume de processamento ou reprodutibilidade rigorosa são requisitos essenciais. Se a resposta for sim, recomenda-se ML clássico ou um modelo pequeno dedicado. Se a resposta for não, a sugestão é utilizar GenAI como ponte, empregando técnicas como few-shot ou RAG e reavaliando a solução quando houver mais dados disponíveis. As etapas são conectadas por setas que representam o caminho das decisões, com respostas "sim" e "não" direcionando o fluxo até a recomendação correspondente. Ao final, uma observação informa que cinco critérios — custo, latência, reprodutibilidade, precisão e disponibilidade de dados — servem como base para orientar essa tomada de decisão.

O desfecho mais útil na prática industrial raramente é puro. O caminho híbrido usa a GenAI como ponte para vencer a partida a frio, quando ainda não há dados rotulados, e prevê a migração para um modelo discriminativo à medida que o volume rotulado cresce e os eixos de custo e reprodutibilidade passam a pesar mais. Registrar essa intenção de migração no momento da decisão evita que a solução de partida se cristalize por inércia. A seção seguinte assume que a GenAI foi escolhida e aborda a primeira decisão correlata, sobre onde o modelo vai rodar.

2. Modelo aberto versus API gerenciada

Esta seção pertence à camada decisória e tem por objetivo que você articule a escolha entre modelo de pesos abertos auto-hospedado e API gerenciada como uma decisão

condicionada pelas demais, e não como uma preferência isolada de infraestrutura. Escolhida a GenAI, esta é a primeira decisão correlata.

Uma vez decidido que a solução é generativa, surge a pergunta de onde o modelo executa. Uma API gerenciada entrega o modelo como serviço: a equipe envia o prompt e recebe a resposta, sem manter infraestrutura. Um modelo de pesos abertos roda no perímetro da própria organização, em servidores ou nuvem privada controlados pela equipe. As duas opções resolvem a mesma tarefa, mas distribuem custo, controle e risco de formas opostas.

GLOSSÁRIO

Modelo de pesos abertos (open-weights)

Modelo cujos parâmetros treinados são distribuídos publicamente, permitindo que a organização o execute, ajuste e audite no próprio ambiente, sem depender do provedor original a cada inferência. Distingue-se de modelo de código aberto, que também libera o código de treino e, por vezes, os dados.

A API gerenciada vence em velocidade de partida. Não exige GPU, MLOps nem provisionamento; o custo por token é previsível em baixo volume; e o provedor cuida de disponibilidade e atualização. Em contrapartida, os dados saem do perímetro da organização a cada chamada, o que pode ser inaceitável sob exigências de privacidade e residência de dados, tema que o módulo M3 trata em detalhe. O custo por token cresce de forma linear com o volume, e a versão do modelo muda sob a aplicação sem aviso, o que afeta diretamente a reprodutibilidade discutida na seção anterior.

A questão da versão merece atenção própria, porque liga esta decisão à reprodutibilidade da anterior. Com uma API gerenciada, o provedor pode atualizar o modelo sob a aplicação, e uma saída que passava nos testes ontem pode mudar hoje sem aviso, o que quebra suites de regressão e dificulta auditoria. Com pesos abertos, a equipe fixa a versão e atualiza quando quiser, ao preço de assumir a manutenção. Para sistemas que precisam de comportamento estável e auditável ao longo do tempo, esse controle de versão pode pesar mais do que qualquer diferença de custo imediato.

O modelo de pesos abertos inverte esses compromissos. Mantém os dados no perímetro, favorecendo conformidade e privacidade; amortiza um custo fixo de infraestrutura que se torna vantajoso em alto volume; e fixa a versão do modelo, devolvendo controle sobre reprodutibilidade. O preço é o esforço: a equipe assume provisionamento de GPU, operação, monitoramento e manutenção, competências que nem todo squad domina no início. A escolha também reduz a dependência de fornecedor, que merece nome próprio.

O esforço de operação merece ser dimensionado com honestidade, porque é o componente que as equipes mais subestimam. Hospedar um modelo de pesos abertos significa servir o modelo com baixa latência sob carga variável, dimensionar GPU para o pico sem desperdiçar no vale, aplicar quantização para caber no hardware disponível, tema que o módulo M4 desenvolve, e montar monitoramento que detecte degradação antes do usuário. Nenhuma dessas tarefas aparece na conta de uma API gerenciada, em que o provedor as absorve. Ignorá-las na decisão produz um orçamento que parece favorável no papel e estoura na operação.

GLOSSÁRIO

Dependência de fornecedor (vendor lock-in)

Situação em que uma solução fica atrelada a um provedor específico, de modo que trocar de fornecedor exige reescrever integrações, revalidar saídas e, por vezes, renegociar custo. Em GenAI, manifesta-se quando prompts, formatos e pós-processamento são calibrados para um modelo proprietário cuja API e versão a organização não controla.

A heurística que organiza os eixos é direta. Em baixo volume, prototipagem ou prova de conceito, a API gerenciada quase sempre vence pelo tempo de partida. Em alto volume sustentado, com dados sensíveis ou exigência regulatória de manter o processamento no país, o modelo de pesos abertos auto-hospedado tende a vencer por custo amortizado, controle e conformidade. Entre os dois extremos existe um ponto de cruzamento de custo que depende do volume projetado, e estimá-lo de forma honesta é parte do dever decisório.

O ponto de cruzamento de custo ilustra bem a interdependência. Em baixo volume, o custo fixo de manter infraestrutura própria se dilui por poucas chamadas e perde para o preço por token de uma API; em alto volume, esse mesmo custo fixo se dilui por milhões de chamadas e passa a vencer com folga. Onde fica o cruzamento depende do preço da API, do custo da infraestrutura e do volume projetado, e por isso a estimativa de volume deixa de ser um detalhe de produto e vira insumo direto da decisão técnica. Uma estratégia comum começa pela API gerenciada na fase de protótipo, quando o volume é baixo e a velocidade importa, e prevê a migração para pesos abertos quando o volume de produção justificar o investimento em infraestrutura.

```
Uma instituição financeira precisa extrair cláusulas de contratos que contêm dados pessoais de clientes. Enviar esses documentos a uma API externa esbarra em obrigações de proteção de dados e em política interna de não exposição de informação sensível. A equipe opta por um modelo de pesos abertos de porte médio, hospedado em nuvem privada, aceitando o custo de operação em troca de manter o processamento dentro do perímetro auditável da instituição.
```

Note como esta decisão não fecha sozinha. Escolher a API gerenciada fixa o custo por inferência que será somado no tópico 5 deste ebook e devolve a reprodutibilidade ao provedor; escolher pesos abertos transfere custo de inferência para custo de infraestrutura e habilita ajuste fino local, tema da próxima seção. A decisão de onde o modelo roda condiciona, portanto, como ele será adaptado à tarefa.

3. In-context learning versus fine-tuning

Esta seção pertence à camada decisória e tem por objetivo que você distinga in-context learning de fine-tuning como mecanismos de adaptação com custos e finalidades diferentes, evitando a confusão de usar um para o que só o outro resolve. Esta é a segunda decisão correlata.

Decidido o modelo e onde ele roda, resta adaptá-lo à tarefa. Existem dois mecanismos centrais, e confundi-los é uma das fontes mais comuns de retrabalho. O in-context learning adapta o comportamento do modelo pela instrução e pelos exemplos colocados no próprio prompt, sem alterar nenhum parâmetro. O fine-tuning ajusta os pesos do modelo a partir de um conjunto de exemplos rotulados, modificando o modelo de forma persistente.

GLOSSÁRIO

In-context learning (aprendizado em contexto)

Capacidade de um modelo generativo de ajustar sua resposta a partir de instruções e de exemplos fornecidos no próprio prompt, sem atualização de pesos. Inclui o regime zero-shot, sem exemplos, e o few-shot, com poucos exemplos demonstrativos. O efeito dura apenas o tempo daquela chamada.

O in-context learning custa pouco para iterar: muda-se a instrução e testa-se de novo, sem treino. Em troca, fica limitado pelo tamanho da janela de contexto, é sensível à forma como a instrução é redigida, e cada token adicional de prompt aumenta o custo

por inferência. O fine-tuning, sobretudo em sua versão leve, com técnicas que ajustam poucos parâmetros, é indicado para fixar estilo, formato e consistência em um domínio estreito, além de reduzir o tamanho do prompt e a latência em produção. O preço é o custo inicial de preparar dados rotulados de qualidade e o risco de overfit e de o conhecimento ficar congelado nos pesos.

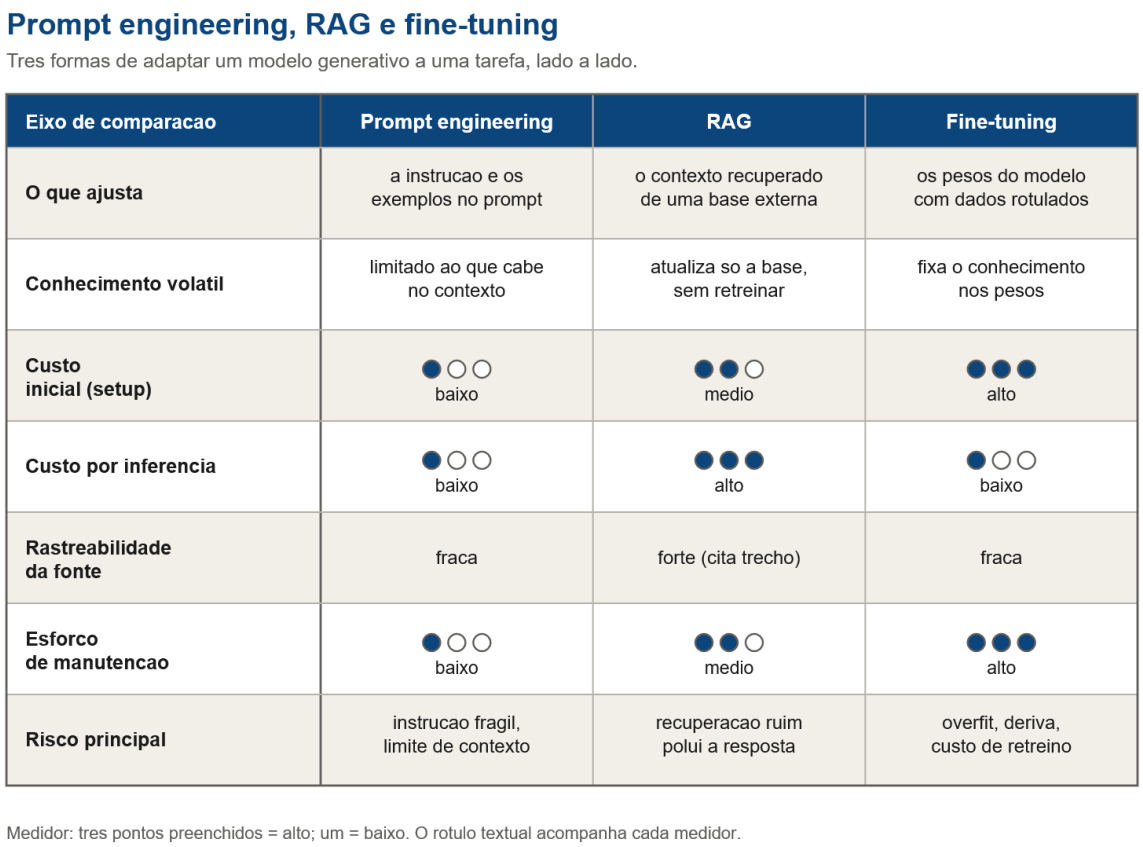
O in-context learning é mais expressivo do que parece à primeira vista. Além de instruir o modelo, ele permite demonstrar o formato de saída esperado com poucos exemplos, restringir o vocabulário e induzir um raciocínio em etapas. Tudo isso, porém, vive dentro da janela de contexto e desaparece ao fim da chamada. A fragilidade aparece quando uma reformulação aparentemente inócua da instrução muda a saída, ou quando a tarefa cresce além do que cabe no contexto. São limites de engenharia de prompt, não defeitos do modelo, e reconhecê-los é o que indica a hora de considerar um mecanismo mais pesado.

> ⚠ ATENÇÃO
>
> A confusão decisória crítica é usar fine-tuning para injetar conhecimento factual ou volátil no modelo. O ajuste de pesos ensina forma e comportamento, como responder em determinado formato ou tom, mas não é o instrumento adequado para manter fatos atualizados, tarefa que cabe à recuperação de contexto. Tentar ensinar fatos por fine-tuning resulta em informação que envelhece, em custo de retreino a cada mudança e em um modelo que ainda assim alucina sobre o que mudou após o treino.

O fine-tuning leve reduz o custo computacional do ajuste, mas não reduz a exigência sobre os dados. Ajustar pesos pede exemplos rotulados que representem bem a tarefa, em quantidade e qualidade suficientes, e essa curadoria costuma ser o gargalo real do projeto, não o treino em si. Um conjunto pequeno ou enviesado produz um modelo que parece melhor nos exemplos vistos e pior no mundo, o que reintroduz o overfitting pela porta dos dados. Por isso o fine-tuning raramente é o primeiro recurso: ele compensa quando a tarefa é estável, o ganho de consistência ou de latência é mensurável, e há dados para sustentá-lo.

O quadro comparativo a seguir põe lado a lado as três formas de adaptar um modelo generativo, incluindo a recuperação de contexto que o tópico 4 deste ebook detalha.

Figura 2 - Matriz comparativa entre prompt engineering, RAG e fine-tuning ao longo de sete eixos de decisão.



Prompt engineering, RAG e fine-tuning

Tres formas de adaptar um modelo generativo a uma tarefa, lado a lado.

Eixo de comparacao	Prompt engineering	RAG	Fine-tuning
O que ajusta	a instrucao e os exemplos no prompt	o contexto recuperado de uma base externa	os pesos do modelo com dados rotulados
Conhecimento volatil	limitado ao que cabe no contexto	atualiza so a base, sem retreinar	fixa o conhecimento nos pesos
Custo inicial (setup)	●○○ baixo	●●○ medio	●●● alto
Custo por inferencia	●○○ baixo	●●● alto	●○○ baixo
Rastreabilidade da fonte	fraca	forte (cita trecho)	fraca
Esforco de manutencao	●○○ baixo	●●○ medio	●●● alto
Risco principal	instrucao fragil, limite de contexto	recuperacao ruim polui a resposta	overfit, deriva, custo de retreino

Medidor: tres pontos preenchidos = alto; um = baixo. O rotulo textual acompanha cada medidor.

Fonte: autor.

Para cego ver: A imagem apresenta a Figura M1.fig-3, intitulada "Matriz comparativa entre prompt engineering, RAG e fine-tuning ao longo de sete eixos de decisão".
A figura consiste em uma tabela comparativa com quatro colunas: Eixo de comparação, Prompt engineering, RAG e Fine-tuning. Cada linha compara as três abordagens em um critério específico. No eixo "O que ajusta", o prompt engineering modifica as instruções e os exemplos inseridos no prompt, o RAG ajusta o contexto recuperado de uma base externa e o fine-tuning modifica os pesos do modelo utilizando dados rotulados. Em "Conhecimento volátil", o prompt engineering é limitado ao conhecimento presente no contexto da conversa, o RAG atualiza as respostas com base em informações recuperadas sem necessidade de novo treinamento, enquanto o fine-tuning fixa o conhecimento aprendido nos parâmetros do modelo. Nos critérios de custo inicial, custo por inferência, esforço de manutenção e risco principal, a intensidade é representada por indicadores visuais de baixo, médio e alto. O prompt engineering apresenta baixo custo inicial, baixo custo por inferência e baixo esforço de manutenção, mas tem como principal risco a fragilidade das instruções e a limitação do contexto. O RAG possui custo inicial e esforço de manutenção intermediários, baixo custo por inferência e oferece rastreabilidade da fonte ao citar diretamente os trechos recuperados, tendo como principal risco a recuperação de informações pouco relevantes. Já o fine-tuning apresenta alto custo inicial e maior esforço de manutenção, baixo custo por inferência e rastreabilidade limitada da fonte, sendo seu principal risco a sobreespecialização do modelo e o custo de novos treinamentos. A tabela utiliza cabeçalho em azul escuro e organiza as informações de forma comparativa, permitindo identificar rapidamente as vantagens, limitações e cenários de aplicação de cada abordagem. Na parte inferior da imagem, consta a indicação "Fonte: autor.".

A heurística de sequenciamento decorre da matriz. Comece pelo prompt engineering, que é o mais barato de testar e revela rapidamente se a tarefa é viável. Acrescente recuperação de contexto quando precisar de conhecimento externo, extenso ou que muda com frequência. Recorra ao fine-tuning apenas quando prompt e recuperação não conseguirem garantir o formato, a consistência ou a latência exigidos, e quando houver dados rotulados estáveis para sustentar o ajuste. Subir de degrau antes da hora gasta esforço que o degrau anterior resolveria.

Um exemplo concreto torna o sequenciamento tangível. Suponha a tarefa de padronizar a extração de campos de laudos técnicos em um formato estruturado. O primeiro recurso é o prompt com poucos exemplos do formato desejado, que costuma resolver a maioria dos casos a custo quase nulo de setup. Se os laudos referenciam normas que mudam, acrescenta-se recuperação de contexto para trazer a norma vigente. Só quando o formato precisa ser rigorosamente consistente em altíssimo volume, e o prompt longo encarece cada chamada, o fine-tuning de um modelo menor passa a compensar, fixando o formato nos pesos e encurtando o prompt.

SAIBA MAIS

WENG, L. Prompt Engineering.
https://lilianweng.github.io/posts/2023-03-15-prompt-engineering, 2023.
O detalhamento de regimes de treino, fine-tuning supervisionado em prática e técnicas de alinhamento é tratado no curso de LLMs da Fase 2 do programa.

A escolha do mecanismo de adaptação define, em boa medida, de onde virão os fatos que o modelo usa. Quando o conhecimento é externo e mutável, a decisão se desloca para a próxima seção, sobre recuperar contexto ou confiar no que o modelo já carrega.

4. RAG versus contexto fechado

Esta seção pertence à camada decisória e tem por objetivo que você decida entre fundamentar a geração em contexto recuperado de uma base externa (RAG) ou confiar no conhecimento já contido no modelo, ponderando volatilidade, rastreabilidade e custo de manutenção. Esta é a terceira decisão correlata.

Toda resposta de um modelo generativo se apoia em algum conhecimento. A decisão aqui é sobre a origem desse conhecimento. No contexto fechado, a resposta vem apenas do que o modelo aprendeu durante o treino, fixado em seus pesos. Na geração aumentada por recuperação, conhecida como RAG, o sistema busca trechos relevantes em uma base externa e os injeta no contexto antes de gerar a resposta, fundamentando a saída em fontes consultáveis.

O contexto fechado dispensa qualquer infraestrutura de recuperação e responde bem sobre conhecimento estável e geral. Em troca, esse conhecimento envelhece na data de corte do treino, não oferece atribuição de fonte e tende a alucinar com mais frequência sobre fatos de nicho ou proprietários, que o modelo nunca viu. O RAG corrige esses três pontos: atualiza-se trocando a base, sem retreinar o modelo; cita o trecho que fundamentou a resposta, o que sustenta auditoria e se conecta à lente de transparência do módulo M5; e reduz a alucinação sobre fatos do domínio ao ancorá-los em documentos reais.

Vale entender, em alto nível, de que partes o RAG é feito, porque cada uma é um ponto de falha e um eixo de custo. Os documentos da base são fragmentados em trechos, convertidos em embeddings e armazenados em um índice; na consulta, a pergunta também vira embedding, recupera os trechos mais próximos e, opcionalmente, passa por um reordenamento antes de entrar no contexto do modelo. Fragmentar mal, escolher

embeddings inadequados ao domínio ou recuperar trechos demais degrada a resposta tanto quanto um modelo fraco. O RAG, portanto, não é um interruptor que se liga, e sim um pipeline que se calibra e se mantém.

Nem toda tarefa justifica esse pipeline. Quando o conhecimento necessário é estável, cabe no prompt e não exige atribuição de fonte, o contexto fechado entrega a resposta com menos partes móveis, menor latência e nenhum custo de índice. Montar RAG nesse caso adiciona complexidade que não se paga. A decisão, portanto, não é qual técnica é superior em abstrato, e sim se as propriedades que o RAG acrescenta, como frescor, escala de conhecimento e rastreabilidade, são exigidas pela tarefa concreta. Quando não são, a simplicidade do contexto fechado é uma vantagem, e não uma limitação.

> Uma área de conformidade precisa responder dúvidas sobre normas técnicas que são revistas a cada poucos meses. Um sistema de contexto fechado responderia com base na versão vigente na data de treino, sem saber das revisões recentes e sem indicar a fonte. Com RAG sobre o repositório atualizado de normas, cada resposta cita o artigo consultado e reflete a versão corrente, e atualizar o sistema significa apenas reindexar os documentos novos.

O RAG também cobra seu preço. Acrescenta um pipeline de recuperação cuja qualidade depende de como os documentos são fragmentados, de quais embeddings indexam os trechos e de como o índice é consultado. Uma recuperação ruim injeta contexto irrelevante e degrada a resposta, em vez de melhorá-la. Há ainda o custo contínuo de manter o índice atualizado à medida que a base muda.

> Uma armadilha recorrente é supor que RAG elimina a alucinação. A recuperação reduz a alucinação sobre fatos do domínio, mas não a anula: quando o passo de recuperação falha e traz o trecho errado, ou nenhum trecho útil, o modelo ainda pode gerar uma resposta confiante e incorreta a partir do contexto pobre. Confiar cegamente no RAG sem avaliar a qualidade da recuperação apenas desloca o ponto de falha, em vez de removê-lo.

A relação com a seção anterior merece ser explicitada, porque concentra a confusão decisória mais cara da lente criativa. Quando o problema é manter fatos atualizados, a resposta é RAG, e não fine-tuning; quando o problema é ajustar forma, estilo ou formato de saída, a resposta é fine-tuning, e não RAG. Fatos pedem recuperação; comportamento pede ajuste de pesos. A heurística de escolha entre RAG e contexto fechado segue a mesma lógica: prefira RAG quando o conhecimento é volátil, extenso ou proprietário e quando a atribuição de fonte importa; fique no contexto fechado quando o conhecimento é estável, pequeno e cabe no próprio prompt.

A atribuição de fonte que o RAG oferece tem valor que extrapola a qualidade da resposta. Em domínios regulados, citar o documento que fundamentou cada afirmação é parte

da exigência de transparência que o módulo M5 trata, e também facilita a validação: um revisor humano confere a resposta contra a fonte sem precisar reexecutar o modelo. O contexto fechado não oferece esse rastro, o que o torna mais barato de operar e mais difícil de auditar. Essa troca entre custo de operação e auditabilidade reaparece em quase todas as decisões do módulo, e raramente tem resposta única.

> GAO, Y. et al. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997, 2023. A implementação instrumentada de pipelines de recuperação, estratégias de chunking e avaliação de retrievers é tratada no curso de LLMs da Fase 2 do programa.

As quatro decisões já percorridas, sobre classe de solução, local de execução, mecanismo de adaptação e origem do conhecimento, convergem para uma pergunta única que a seção final responde: onde, afinal, mora o custo, e como traduzir tudo isso em compromissos defensáveis de projeto.

5. Alocação de custo e fechamento decisório

Esta seção pertence à camada decisória e tem por objetivo que você decida onde alocar custo em uma solução generativa e conecte a decisão GenAI versus ML clássico ao ciclo de um projeto, traduzindo-a em compromissos de cronograma, custo e validação defensáveis perante stakeholders não técnicos.

A decisão sobre custo costuma ser tratada como detalhe de engenharia tardio, quando deveria ser parte do desenho. Uma solução generativa distribui custo por várias fontes, e enxergá-las em conjunto é o que separa uma estimativa honesta de uma surpresa em produção. O custo por inferência depende de tokens, volume de chamadas e preço; o custo inicial cobre setup, eventual fine-tuning e construção de índice; o custo de manutenção acompanha deriva temporal, reindexação e reavaliação contínua; e o custo de risco, muitas vezes esquecido, embute alucinação, propriedade intelectual e conformidade.

GLOSSÁRIO

Custo de inferência

Custo incorrido a cada execução do modelo em produção, medido em tokens processados, tempo de computação ou chamadas a uma API. Diferencia-se do custo de treino ou de setup, que ocorre uma vez, e cresce com o volume de uso, o que o torna o componente dominante do custo total em aplicações de alto tráfego.

Estimar custo não exige um número exato, e sim a ordem de grandeza certa e os termos que dominam. Em uma aplicação de alto tráfego, o custo por inferência tende a dominar tudo, de modo que reduzir o tamanho médio do prompt ou trocar por um modelo menor tem impacto maior do que otimizar o setup. Em uma aplicação de baixo tráfego com base de conhecimento grande, o custo de manter o índice atualizado pode superar o custo das próprias chamadas. Identificar qual termo domina antes de otimizar evita o erro clássico de poupar onde não importa, e é uma das traduções mais úteis que a equipe técnica leva ao stakeholder.

A validação de saídas generativas merece um cuidado que o classificador clássico não exige. Como não há um único rótulo certo, avalia-se com conjuntos de exemplos representativos, com critérios explícitos de qualidade e, quando cabe, com um modelo juiz que pontua as respostas, sempre lembrando que esse juiz também erra e precisa ser calibrado contra julgamento humano. Suites de regressão guardam casos que já funcionaram e alertam quando uma mudança de prompt, de modelo ou de base os quebra. Sem esse aparato, a qualidade da solução vira impressão, e impressão não sustenta um compromisso perante stakeholders.

As cinco decisões do módulo não são independentes; elas formam uma cadeia em que cada escolha fixa um termo do custo total. Decidir por GenAI em vez de ML clássico define o regime inteiro de gasto recorrente. Escolher API gerenciada em vez de pesos abertos converte o custo em preço por token, previsível mas linear no volume; escolher pesos abertos converte-o em custo de infraestrutura, fixo mas dependente de operação. Adotar RAG acrescenta o custo de manter um índice; adotar fine-tuning antecipa um custo de setup e de retreino. Ler as decisões como uma cadeia, e não como caixas isoladas, é o núcleo da competência desta camada.

⚠ ATENÇÃO

A armadilha mais cara é estimar o custo de produção a partir do custo de protótipo. Um piloto com baixo volume e prompts curtos sugere um gasto modesto que se multiplica quando o tráfego real chega, os prompts crescem e a janela de contexto se enche. Apresentar a um stakeholder o custo do protótipo como se fosse o custo de operação gera compromissos que o orçamento não sustenta, e mina a confiança na decisão técnica quando a conta real aparece.

O fechamento da lente criativa consiste em traduzir a decisão GenAI versus ML clássico em três compromissos que um stakeholder não técnico entende e cobra. No cronograma, a GenAI com partida a frio prototipa rápido, mas a construção da avaliação adiciona tempo que precisa ser previsto; o ML clássico exige tempo de rotulagem antes de qualquer resultado. No custo, apresente faixas de custo total de propriedade, e não números pontuais, separando claramente o regime de protótipo do regime de produção. Na validação, reconheça que saídas não determinísticas exigem instrumentos diferentes dos de um classificador: conjuntos de avaliação, uso cuidadoso de avaliação por modelo juiz e suites de regressão, organizados pelo checklist T1.3.

Traduzir para o stakeholder não significa simplificar a ponto de distorcer, e sim falar na moeda de cada interlocutor. Quem cuida do orçamento quer previsibilidade de custo e o ponto em que ele escala; quem responde pelo produto quer latência aceitável e qualidade percebida; quem responde por risco quer auditabilidade e conformidade. A mesma decisão GenAI versus ML clássico se apresenta de forma diferente a cada um, sem mudar de conteúdo, e a competência decisória inclui sustentar a escolha em todas essas conversas com o mesmo conjunto de evidências.

Esses compromissos se registram nos três instrumentos do módulo. A matriz de decisão T1.1 documenta a escolha GenAI versus ML clássico ao longo dos cinco eixos, com justificativa por eixo; a matriz de riscos T1.2 mapeia alucinação, viés, propriedade intelectual, custo e deriva temporal, com probabilidade, severidade e mitigação; e o checklist T1.3 verifica se a avaliação das saídas foi montada antes do deploy. Preencher os três de forma articulada transforma a decisão de uma opinião em um compromisso defensável, que é exatamente o que os encontros 5 e 6 da Formação 1 cobram quando a escolha precisa virar cronograma e validação. Uma decisão técnica que não se traduz em compromisso de projeto permanece incompleta, por mais correta que seja no plano abstrato.

Por fim, trate a decisão como revisitável. O ponto de cruzamento entre as alternativas se move quando o volume cresce, quando os dados rotulados se acumulam ou quando a regulação muda. A decisão certa hoje pode merecer reabertura amanhã, e registrar as premissas que a sustentaram é o que torna essa reabertura barata.

6. Conclusão

A camada decisória deslocou a lente criativa de uma pergunta sobre tecnologia para uma pergunta sobre escolha. A decisão GenAI versus ML clássico não mede sofisticação, e sim a adequação de uma classe de solução à tarefa, ao menor custo de risco. Tudo parte de uma única pergunta âncora, sobre a natureza da saída, e se afina nos cinco eixos da matriz T1.1: custo, latência, reprodutibilidade, precisão e disponibilidade de dados.

As cinco decisões do módulo não são independentes; elas formam uma cadeia. A escolha entre GenAI e ML clássico define o regime de gasto recorrente; a escolha entre modelo aberto e API gerenciada converte esse gasto em custo de infraestrutura ou em preço por token; a escolha entre in-context learning e fine-tuning define como o modelo é adaptado; a escolha entre RAG e contexto fechado define de onde vêm os fatos; e a alocação de custo amarra todas as anteriores. Ler essas decisões como uma cadeia, e não como caixas isoladas, é o núcleo da competência M1.C3.

O que se espera ao final não é dominar cada técnica, e sim justificar uma decisão defensável. Uma escolha técnica só se completa quando vira compromisso de cronograma, custo e validação, traduzível para stakeholders não técnicos e registrável nos instrumentos do módulo: a matriz T1.1, a matriz de riscos T1.2 e o checklist T1.3. E, porque o ponto de equilíbrio entre as alternativas se move com volume, dados e regulação, toda decisão deve ser tratada como revisitável, com suas premissas registradas. É esse repertório que os encontros 5 e 6 da Formação 1 cobram quando a solução precisa ser desenhada.

REFERÊNCIAS

LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020. Preprint em https://arXiv.org/abs/2005.11401.

GAO, Y. et al. Retrieval-Augmented Generation for Large Language Models: A Survey. Preprint em https://arxiv.org/abs/2312.10997, 2023.

OUYANG, L. et al. Training Language Models to Follow Instructions with Human Feedback (InstructGPT). NeurIPS, 2022. Preprint em https://arXiv.org/abs/2203.02155.

VASWANI, A. et al. Attention Is All You Need. NeurIPS, 2017. Preprint em https://arXiv.org/abs/1706.03762.

Bibliografia complementar

WENG, L. Prompt Engineering. Disponível em: https://lilianweng.github.io/posts/2023-03-15-prompt-engineering. 2023.

RAFAILOV, R. et al. Direct Preference Optimization. NeurIPS, 2023. Preprint em https://arXiv.org/abs/2305.18290.

UNIÃO EUROPEIA. Regulamento (UE) 2024/1689 (EU AI Act). Artigos sobre GPAI (51 a 55). Texto integral em eur-lex.europa.eu.

BRASIL. Senado Federal. Projeto de Lei n. 2338/2023. Marco regulatório da IA no Brasil. Texto integral em portais públicos do Senado.

Industr.IA

Residência em TIC 49



Executor iRede

Executor Atlântico

Coordenador Softex

Financiador MCTI Futuro

Ministério da Ciência, Tecnologia e Inovação

Governo do Brasil — Do lado do povo brasileiro

Projeto apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991.